

Pruebas de los requerimientos

Traza de requerimientos

La trazabilidad en los requerimientos puede presentarse partiendo del SRS (Software Requirement Specification) y, si es posible establecer de forma clara donde se ha originado cada uno de ellos, servirá luego para poder referencia en las futuras etapas del desarrollo.

La trazabilidad remota hace referencia a las fases previas al desarrollo, es decir, referenciando el origen o fuente del requerimiento.

La trazabilidad futura apunta a las fases posteriores del desarrollo, para lo que es imprescindible que cada requerimiento tenga un nombre o referencia única.

El momento en que el producto de software pasa a ser operado y mantenido, la trazabilidad remota cobra más importancia, ya que al llevar a cabo modificaciones en el diseño o el código, se vuelve fundamental poder identificar los requerimientos afectados por dicho cambio para llevar a cabo la actualización correspondiente.

Recordemos que podemos emplear los siguientes tipos de trazabilidad:

- **Trazabilidad hacia atrás desde los requerimientos:** podemos conocer porque cada requerimiento en el SRS existe.
- **Trazabilidad hacia adelante hasta los requerimientos:** todos los documentos que preceden al SRS pueden referenciarse al SRS.
- **Trazabilidad hacia atrás hasta los requerimientos:** cada comportamiento del software refiere explícitamente aquellos requerimientos que satisface.

La trazabilidad hacia adelante desde los requerimientos: podemos entender cuáles componentes del software satisfacen cada requerimiento.

Revisión y aceptación

La prueba de aceptación es realizada por los usuarios para verificar que el sistema se ajusta a sus requerimientos. Los casos de prueba se basan en la especificación de requerimientos y es una técnica de caja negra.

Los pasos o actividades de este proceso son:

- Determinar el rol del usuario
- Definir los criterios de aceptación
- Desarrollar un plan de aceptación

- Ejecutar el plan de aceptación
- Determinar decisiones de aceptación.

Esta prueba final se lleva a cabo con datos proporcionados por el cliente, ya no son datos ficticios, y puede suceder que estas diferencias entre datos reales y simulados, dispare en las pruebas errores del sistema, o se detecte errores en los requerimientos.

Prueba de Aceptación basada en los requerimientos.

Según el [International Software Testing Qualifications Board](#), las pruebas de aceptación son pruebas formales con respecto a las necesidades del usuario, los requisitos y los procesos comerciales realizados para determinar si un sistema cumple o no con los criterios de aceptación y para permitir que el usuario, los clientes u otra entidad autorizada determinen si aceptan o no el sistema.

Cuando referenciamos las pruebas de aceptación basadas en los requerimientos, mencionamos que los requerimientos que documentamos puedan ser probados, es decir que basándose en dichos requerimientos, se diseñarán las pruebas correspondientes para comprobar si el sistema los cumple. Este tipo de prueba son pruebas de validación y no de prueba de defectos, decimos con esto que lo que se busca es demostrar si el sistema ha implementado en forma correcta los requerimientos establecidos.

Por ejemplo, si un requerimiento establecía que “el usuario podrá consultar los datos de un producto indicando un dato del mismo”, las posibles pruebas podrían ser: realizar una búsqueda de un producto existente en la base de datos, realizar una búsqueda de un producto inexistente en la base de datos, realizar una búsqueda cuyo dato coincida con más de un producto de la base de datos.

Validación y verificación de requerimientos

El desarrollo que se lleva a cabo debe formal y técnicamente ser correcto, y satisfacer los requisitos del sistema. Esta comprobación se llevará a cabo durante cada etapa del proceso de desarrollo.

La tarea de revisión y pruebas se denomina V&V (verificación y validación), e iniciará con la comprobación de los requerimientos, luego en el diseño, el código, y finalmente en el producto final que se desarrolle.

Veamos cómo define Boehm (1979) estos términos para comprender la diferencia¹:

- Validación: ¿Estamos construyendo el producto correcto?
- Verificación: ¿Estamos construyendo el producto correctamente?

La diferenciación primordial que radica en estos conceptos es que el primero, la validación, esta refiriendo a demostrar que la definición de los requerimientos que hemos generado realmente se corresponda con el sistema que el usuario o el cliente necesitan o esperan. Por ello es muy importante que el cliente valide esa especificación de requerimientos que generemos, corroborando que no contenga errores o inconsistencias.

Por otro lado, la verificación implica comprobar que el producto de software está de acuerdo a la especificación de requerimientos que fue generada previamente.

Estrategias de prueba (testing)

La estrategia de prueba define el enfoque que se empleará y los objetivos generales de las actividades de prueba que se llevarán a cabo. En esta definición se deben incluir las fases de prueba que se deben seguir y también qué clases de prueba se realizarán en cada una.

Los tipos de fases de prueba son:

- **De unidad:** prueba un módulo o componente. Generalmente lo realiza quien lo construyó.
- **De integración:** se basa en revisar las respuestas de grupos de módulos interconectados, con el objetivo de detectar fallos en la interacción entre los componentes.
- **De sistema:** se prueba el sistema en un ambiente similar al real. Acá debe considerarse que el sistema desarrollado puede ser parte de un sistema mayor.

Las opciones de prueba que pueden ejecutarse en el sistema son varias, por ejemplo:

- de recuperación,
- de seguridad,
- de resistencia o stress,
- de rendimiento,
- de seguridad,
- de aceptación,

¹ Boehm, B. W. (1979). *Software engineering; R & D Trends and defense needs. In research Directions in Software Technology* (P. Wegner, ed.). Cambridge, MA: MIT Press.

- de regresión

La estrategia debería contemplar qué técnicas y herramientas de pruebas se van a utilizar, así como los criterios de satisfacción y finalización que se emplearán. La fase de prueba se centra fundamentalmente en asegurarse que el sistema cumpla con todos los requerimientos definidos. Por ello, al determinar la estrategia de prueba, lo que se llevará a cabo y los criterios que se aplicarán para cubrir las pruebas se basarán en lo que se determinó en la definición de requisitos.

Introducción al concepto de testing ágil

El testing ágil es una práctica de prueba para un proyecto utilizando metodologías de desarrollo de software ágil, incorporando técnicas y métodos, como la programación extrema (XP), tratando el desarrollo como el cliente de las pruebas y enfatizando el paradigma de diseño de prueba primero, según indica el [International Software Testing Qualifications Board](#), las pruebas son obtenidas por el equipo de desarrollo, ya que mientras llevan a cabo el análisis, conjuntamente se van diseñando las pruebas del sistema.

Al aplicar un desarrollo ágil, el equipo mantiene una comunicación continua entre los integrantes, definiendo los requisitos, las pruebas y el comportamiento esperado del producto. Además, se manifiesta el concepto de roles, donde se intenta que cada miembro posea un buen número de roles para eliminar la dependencia entre individuos, promoviendo siempre a todos los principios de calidad en sus actividades diarias. En este marco, en un equipo ágil todos los individuos tienen habilidades de probadores.

Un probador ágil se caracteriza por tener, además de habilidades técnicas en testing, destreza para integrar las necesidades del interesado al equipo desarrollador, definiendo las historias del usuario en requisitos y transformándolas en parámetros de prueba cuando se codifiquen. El rol de tester o probador sobresale por colaborar con todos los integrantes del equipo.

Los roles que giran alrededor de la prueba ágil son:

Tester: persona del equipo que puede desempeñar varios roles en un proyecto de software. Tiene habilidades y destrezas para el desarrollo y ejecución de pruebas.

Interesado: persona o entidad que se apoya en un analista con el fin de proponer una solución a un problema. Elabora el discurso de los requisitos, que encaminan la solución.

Equipo ágil: conjunto de personas expertas en áreas de software y en el dominio del negocio que trabajan orientadas hacia un mismo fin en pequeños ciclos de tiempo, llamados iteraciones.

La filosofía de calidad toma forma en el desarrollo de software cuando todos los miembros del equipo son probadores, pues desaparece la fase de pruebas, que forma parte de los métodos tradicionales.

El diseño de pruebas, como tal, y el uso de las herramientas adecuadas se vuelven vitales para asegurar el principio de calidad. La planeación y el diseño de pruebas es importante a la hora de distribuir tiempo en testing particular, arreglar bugs y realizar otro tipo de pruebas como las mencionadas previamente (de aceptación, de regresión, etc.).

El factor de éxito del testing ágil es saber automatizar las pruebas de regresión, que aseguran calidad y precisión sobre todo el comportamiento del sistema. Además, se deberían automatizar las pruebas de aceptación que aseguran al cliente la funcionalidad que requirió en un principio.